Hallazgos del SOLID

1. S (Responsabilidad Única)

- Evidencia: En la revisión de la Etapa 4, CU-09 (acordarCambioModalidad) hacía que el servicio revalidara plazo y compatibilidad, y CU-11b (liquidarPagoTutor) le hacía consultar la elegibilidad en Sesión, Disputa y Pago.
- Impacto: El servicio tendría que cambiar cada vez que cambien BR-0013 o las reglas de liquidación (BR-0005, BR-0017).
- Cambio: La regla del plazo pasa a Sesion y AcuerdoDeModalidad. La elegibilidad de liquidación pasa al servicio de dominio PoliticaDeLiquidacion. El cálculo de penalización y reembolso queda en PoliticaDeReembolso, y SesionApplicationService solo coordina y persiste.

2. O (Abierto/Cerrado), propuesta

- Evidencia: PoliticaDeReembolso decide el monto según la causa: cancelación tardía, ausencia del tutor, ausencia del estudiante o corte de conexión (BR-0004, 0008, 0009, 0010). Si cada causa es un if, cada política nueva obliga a editar la clase.
- Impacto: Una excepción nueva del marketplace rompería código que ya funciona.
- Cambio: PoliticaDeReembolso se define como una interfaz con una implementación por causa, y una política nueva se agrega sin tocar las existentes. Está marcado como propuesta porque el diagrama actual muestra una sola clase.

3. L (Sustitución de Liskov)

- Evidencia: SqlReservaRepository, SqlSesionRepository, SqlTutorRepository, SqlPagoRepository y SqlDisputaRepository implementan cada una la interfaz de su Aggregate.
- Impacto: Si alguna implementación alterara el contrato (por ejemplo, devolviendo un Aggregate incompleto), el dominio fallaría.
- Cambio: Los contratos se definen solo con tipos del dominio, y cualquier implementación, incluida una en memoria para pruebas, puede sustituir a otra sin cambiar el comportamiento.

4. I (Segregación de Interfaces)

- Evidencia: Los Handlers de lectura (ConsultarDashboardSemanalQueryHandler, ConsultarPerfilTutorQueryHandler) solo necesitan consultar datos.
- Impacto: Si usaran los Repository, dependerían de métodos de escritura como guardar.
- Cambio: Las lecturas usan DashboardReadModel y PerfilTutorReadModel, y los Commands usan Repository y UnitOfWork. Además hay un Repository por Aggregate, no una interfaz gigante.

5. D (Inversión de Dependencias)

- Evidencia: ReservaApplicationService necesita cobrar y guardar, lo que podría atarlo a un proveedor de pagos y a SQL.
- Impacto: Pruebas unitarias difíciles y cambios tecnológicos costosos.
- Cambio: El servicio depende de PasarelaPagoExterna, los Repository y UnitOfWork. Las implementaciones (AdaptadorPasarelaPago, Sql*Repository, SqlUnitOfWork) viven en Infraestructura.